

Mục lục

1 ĐẶT VẤN ĐỀ	5
1.1 Bối cảnh	5
1.2 Vấn đề nghiên cứu	6
1.2.1 Phân quyền là một khía cạnh cắt ngang	6
1.2.2 Ba đặc thù khiến RBAC khó mô-đun hóa	6
1.2.3 Khoảng cách giữa mô hình và mã nguồn	7
1.3 Khoảng trống nghiên cứu	7
1.4 Mục tiêu và câu hỏi nghiên cứu	8
1.4.1 Phát biểu bài toán	8
1.4.2 Câu hỏi nghiên cứu	8
1.5 Đóng góp của luận văn	9
1.6 Phạm vi và giới hạn	9
1.7 Bố cục luận văn	10
2 CƠ SỞ LÝ THUYẾT	11
2.1 Thiết kế hướng miền	11
2.1.1 Ngôn ngữ chung và ngữ cảnh giới hạn	11
2.1.2 Các khuôn mẫu chiến thuật	12
2.1.3 Kiến trúc phân tầng	12
2.2 Ngôn ngữ chuyên biệt miền	13
2.2.1 Khái niệm và phân loại	13
2.2.2 Ba thành phần của một ngôn ngữ	13
2.2.3 Ngôn ngữ chuyên biệt miền dựa trên chú giải cho DDD	13
2.3 Kiểm soát truy cập dựa trên vai trò	14
2.3.1 Mô hình cơ sở $RBAC_0$	14
2.3.2 $RBAC_1$: phân cấp vai trò	14
2.3.3 $RBAC_2$: ràng buộc	15
2.3.4 Biểu diễn RBAC bằng UML và OCL	15
2.3.5 Phân quyền đa miền	16
2.4 Chuẩn mô hình hóa hình thức và công cụ	16
2.4.1 Cơ chế profile của UML	16

2.4.2	Ngôn ngữ ràng buộc đối tượng	16
2.4.3	Công cụ kiểm chứng USE	17
2.4.4	PlantUML như một cú pháp văn bản của sơ đồ UML	17
2.5	Hợp nhất khía cạnh quan tâm và kiến trúc đích	17
2.5.1	Cơ chế hợp nhất của UDML	17
2.5.2	Kiến trúc vi dịch vụ phân tầng TMSA	17
2.5.3	Nền tảng cài đặt NestJS	18
2.6	Tổng kết chương	18
3	PHƯƠNG PHÁP ĐỀ XUẤT	19
3.1	Tổng quan phương pháp	19
3.1.1	Nguyên tắc thiết kế	19
3.1.2	Kiến trúc tổng thể	19
3.2	Ngôn ngữ đặc tả chính sách RBAC	20
3.2.1	Cú pháp trừu tượng: metamodel RBAC	20
3.2.2	Cú pháp cụ thể: profile trên sơ đồ lớp	21
3.2.3	Ngữ nghĩa tĩnh: tập bất biến	22
3.3	Bước 1 – Phân tích cú pháp	23
3.3.1	Mục tiêu và đầu ra	23
3.3.2	Xử lý từng nguồn đặc tả	24
3.3.3	Vị trí nguồn và chẩn đoán lỗi	24
3.4	Bước 2 – Hợp nhất mô hình	24
3.4.1	Mô hình miền hợp nhất	24
3.4.2	Tham chiếu chéo giữa các khía cạnh	25
3.4.3	Bất biến sau hợp nhất	25
3.5	Bước 3 – Làm giàu mô hình	25
3.5.1	Ba phép biến đổi trên khía cạnh cấu trúc	25
3.5.2	Lan truyền quyền theo phân cấp vai trò	26
3.6	Bước 4 – Kiểm chứng ràng buộc	27
3.6.1	Quy trình đánh giá	27
3.6.2	Ranh giới giữa hai thời điểm kiểm chứng	27
3.7	Bước 5 – Sinh mã theo mẫu	28
3.7.1	Ánh xạ khái niệm sang sản phẩm mã	28
3.7.2	Ba nguyên tắc của cơ chế sinh	28
3.8	Các tính chất của phương pháp	29
3.9	Tổng kết chương	29

Danh sách hình vẽ

3.1 Tổng quan phương pháp ở mức khái niệm	20
3.2 Metamodel chính sách RBAC (cú pháp trừu tượng)	21

Danh sách bảng

1.1	Đối chiếu các công trình liên quan theo năm tiêu chí	8
2.1	Các khuôn mẫu chiến thuật của DDD được sử dụng trong luận văn	12
3.1	Ánh xạ giữa cú pháp trừu tượng và cú pháp cụ thể	22
3.2	Tập bất biến định nghĩa ngữ nghĩa tĩnh của ngôn ngữ	23
3.3	Ánh xạ khái niệm mô hình sang sản phẩm mã	28

Chương 1

ĐẶT VẤN ĐỀ

1.1 Bối cảnh

Phần mềm nghiệp vụ hiện đại phải xử lý những quy tắc ngày càng phức tạp, thay đổi liên tục và gắn chặt với ngữ cảnh tổ chức sử dụng nó. Trước thực tế đó, Thiết kế hướng miền (*Domain-Driven Design* - DDD) do Evans (2003) đề xuất chủ trương đặt **mô hình miền** vào trung tâm của quá trình phát triển: thay vì bắt đầu từ cơ sở dữ liệu hay giao diện, đội phát triển cùng chuyên gia nghiệp vụ xây dựng một mô hình khái niệm phản ánh đúng ngôn ngữ và quy tắc của lĩnh vực, rồi tổ chức toàn bộ mã nguồn xoay quanh mô hình đó. Các khái niệm nền tảng của DDD - ngôn ngữ chung (*ubiquitous language*), ngữ cảnh giới hạn (*bounded context*) và các khuôn mẫu chiến thuật như thực thể, đối tượng giá trị, gốc tụ hợp, kho lưu trữ, dịch vụ miền, sự kiện miền - đã trở thành vốn từ chung của cộng đồng kỹ nghệ phần mềm (Vernon, 2013).

Song song với đó, kiến trúc vi dịch vụ (*microservices*) trở thành lựa chọn phổ biến cho các hệ thống quy mô lớn. Điểm giao thoa quan trọng giữa hai xu hướng là nguyên tắc phân rã hệ thống theo ngữ cảnh giới hạn: mỗi ngữ cảnh giới hạn trở thành một dịch vụ có mô hình miền và vòng đời phát triển riêng (Newman, 2021; Richardson, 2018). Khi hệ thống được tách thành nhiều dịch vụ, nhu cầu bảo đảm *tính nhất quán của chính sách phân quyền* giữa các dịch vụ càng trở nên cấp thiết: cùng một vai trò người dùng phải được hiểu và thực thi thống nhất ở mọi nơi trong hệ thống.

Trong mạch nghiên cứu về sinh mã tự động cho DDD, nhóm nghiên cứu tại Trường Đại học Công nghệ, Đại học Quốc gia Hà Nội đã phát triển một hướng tiếp cận dùng **ngôn ngữ chuyên biệt miền dựa trên chú giải** (*annotation-based domain-specific language* - aDSL). Hướng này khởi đầu với DCSL cho khía cạnh cấu trúc của mô hình miền (Le et al., 2018, 2020), được mở rộng bởi AGL cho khía cạnh hành vi (Dang et al., 2023; Le et al., 2023), được tổng quát hóa bởi cơ chế hợp nhất nhiều khía cạnh quan tâm UDML (Le and Dang, 2025), và có kiến trúc đích là mô hình vi dịch vụ phân tầng TMSA (Le et al., 2025). Đặc điểm chung của mạch nghiên cứu này là: mô hình miền được đặc tả bằng một ngôn ngữ hình thức ở mức mô hình, sau đó phần mềm thực thi được sinh tự động từ đặc tả đó.

Luận văn này đặt vấn đề tiếp nối mạch nghiên cứu trên với một khía cạnh quan tâm chưa được xử lý: **kiểm soát truy cập dựa trên vai trò** (*Role-Based Access Control* -

RBAC).

1.2 Vấn đề nghiên cứu

1.2.1 Phân quyền là một khía cạnh cắt ngang

RBAC (Sandhu et al., 1996) là mô hình kiểm soát truy cập được sử dụng rộng rãi nhất trong phần mềm doanh nghiệp: quyền truy cập không gán trực tiếp cho người dùng mà gán cho vai trò, người dùng nhận quyền thông qua vai trò được giao. Tuy phổ biến ở mức khái niệm, RBAC lại thường được *cài đặt thủ công* ở tầng hạ tầng của hệ thống: các đoạn mã kiểm tra quyền nằm rải rác trong bộ điều khiển, trong lớp dịch vụ, đôi khi cả trong mô hình miền.

Theo thuật ngữ của lập trình hướng khía cạnh, RBAC là một **khía cạnh cắt ngang** (*cross-cutting concern*) điển hình, biểu hiện qua hai triệu chứng:

- **Rải rác** (*scattering*): logic của cùng một chính sách phân quyền bị phân tán ra nhiều mô-đun. Thêm một vai trò mới kéo theo việc sửa đổi ở nhiều tệp nguồn khác nhau.
- **Trộn lẫn** (*tangling*): trong cùng một đơn vị mã, câu lệnh kiểm tra quyền xen kẽ với logic nghiệp vụ, làm giảm khả năng đọc hiểu và khả năng kiểm thử độc lập của cả hai phần.

Hệ quả thực tiễn là chi phí bảo trì cao và rủi ro an ninh: một điểm kiểm tra bị bỏ sót trong hàng chục điểm rải rác đủ để tạo ra lỗ hổng, trong khi việc rà soát thủ công gần như không khả thi với hệ thống lớn.

1.2.2 Ba đặc thù khiến RBAC khó mô-đun hóa

So với các khía cạnh cắt ngang quen thuộc khác như ghi nhật ký hay quản lý giao dịch, RBAC có ba đặc thù làm cho việc tách nó thành một mô-đun độc lập trở nên khó hơn đáng kể:

1. **Tham chiếu chéo tới mô hình miền.** Một quyền chỉ có ý nghĩa khi gắn với đối tượng nghiệp vụ cụ thể: quyền VIEW_STUDENTS chỉ được hiểu đúng khi biết nó cho phép thao tác *đọc* trên lớp miền Student với phạm vi xác định. Nói cách khác, đặc tả phân quyền buộc phải vượt ranh giới của chính nó để trở vào khía cạnh cấu trúc.
2. **Tính không cục bộ do phân cấp vai trò.** Trong RBAC₁, vai trò cấp cao kế thừa toàn bộ quyền của vai trò cấp thấp hơn. Do đó quyền hiệu dụng của một vai trò không đọc được từ khai báo cục bộ của vai trò đó mà phải tính qua bao đóng trên đồ thị kế thừa.
3. **Ràng buộc phạm vi toàn hệ thống.** Các ràng buộc tách nhiệm tĩnh (SSD), tách nhiệm động (DSD) và giới hạn số lượng người giữ vai trò được phát biểu trên toàn bộ tập vai trò và tập người dùng. Đây là các mệnh đề bậc cao mà một điểm kiểm tra cục bộ trong mã nguồn không thể bảo đảm.

1.2.3 Khoảng cách giữa mô hình và mã nguồn

Vấn đề thứ ba mang tính phương pháp luận. Trong thực tế, chính sách phân quyền thường được mô tả trong tài liệu thiết kế bằng ngôn ngữ tự nhiên hoặc bảng biểu, rồi được lập trình viên hiện thực hóa thủ công. Không có cơ chế nào bảo đảm mã nguồn thực thi đúng chính sách đã được phê duyệt, cũng không có cách nào kiểm chứng bản thân chính sách đó có nhất quán hay không *trước khi* viết mã. Sau vài chu kỳ phát triển, tài liệu thiết kế và mã nguồn phân kỳ; câu hỏi “ai thực sự được làm gì trong hệ thống này” chỉ trả lời được bằng cách đọc mã.

1.3 Khoảng trống nghiên cứu

Bài toán nêu trên không hoàn toàn mới. Có ít nhất bốn dòng công trình đã chạm tới nó từ những góc độ khác nhau.

Nền tảng lý thuyết RBAC. Sandhu et al. (1996) định nghĩa họ mô hình tham chiếu $RBAC_0$ – $RBAC_3$, cung cấp ngữ nghĩa hình thức cho vai trò, quyền, phiên làm việc, phân cấp vai trò và các ràng buộc. Ray et al. (2004) đi thêm một bước: biểu diễn ràng buộc RBAC bằng UML kết hợp OCL và trực quan hóa vi phạm bằng mẫu sơ đồ đối tượng. Tuy vậy, các công trình này mô hình hóa RBAC như một hệ thống độc lập để phân tích, chưa gắn nó với mô hình miền của một ứng dụng cụ thể và không đề cập tới việc sinh mã.

An ninh hướng mô hình. SecureUML (Lodderstedt et al., 2002) và bản mở rộng của nó (Basin et al., 2006) là những công trình mở đường: chính sách phân quyền được mô hình hóa bằng một UML profile, sau đó hạ tầng kiểm soát truy cập được sinh tự động. Đây là hướng gần nhất với luận văn về mặt tinh thần. Điểm khác biệt nằm ở chỗ SecureUML gắn với hệ công cụ mô hình hóa nặng và nền tảng sinh mã của thời kỳ đó, không đặt trong khung hợp nhất nhiều khía cạnh của DDD, và không sinh ra scaffolding phân tầng theo ngữ cảnh giới hạn như kiến trúc hiện đại đòi hỏi.

aDSL cho Thiết kế hướng miền. DCSL (Le et al., 2018, 2020) và AGL (Dang et al., 2023) cho thấy hiệu quả của việc đặc tả mô hình miền bằng chú giải rồi sinh mã: khía cạnh cấu trúc và khía cạnh hành vi đều đã có ngôn ngữ đặc tả riêng, có ngữ nghĩa hình thức và có bộ sinh mã tương ứng. Tuy nhiên, phân quyền chưa nằm trong phạm vi của hai ngôn ngữ này.

Hợp nhất khía cạnh quan tâm. UDML (Le and Dang, 2025) đưa ra cơ chế tổng quát để gắn nhiều khía cạnh vào một mô hình miền thống nhất thông qua cấu trúc *Annotable/Annotation*, bảo đảm ba tính chất trực giao, gắn kết và truy vết. Công trình này có nhắc tới kiểm soát truy cập như một khía cạnh khả thi, nhưng dừng ở mức nêu khả năng: chưa có metamodel, chưa có cú pháp đặc tả, chưa có ngữ nghĩa hình thức và chưa có bộ sinh mã cho khía cạnh này.

Bảng 1.1 đối chiếu các dòng công trình theo năm tiêu chí phản ánh trực tiếp yêu cầu của bài toán.

Khoảng trống mà luận văn hướng tới nằm ở dòng cuối cùng của bảng: *chưa có công trình nào đồng thời đặc tả RBAC như một khía cạnh hạng nhất của mô hình miền trong DDD, cho phép kiểm chứng chính sách ở mức thiết kế, và sinh tự động phần mềm thực thi đúng chính sách đó.*

Bảng 1.1: Đối chiếu các công trình liên quan theo năm tiêu chí

Công trình	Trọng tâm	(1)	(2)	(3)	(4)	(5)
Sandhu et al. (1996)	Mô hình tham chiếu RBAC	-	-	✓	-	-
Ray et al. (2004)	Biểu diễn ràng buộc UML/OCL	✓	-	✓	✓	-
Basin et al. (2006)	An ninh hướng mô hình	✓	✓	-	-	✓
Le et al. (2018)	aDSL cấu trúc (DCSL)	✓	✓	-	✓	✓
Dang et al. (2023)	aDSL hành vi (AGL)	✓	✓	-	✓	✓
Le and Dang (2025)	Hợp nhất khía cạnh (UDML)	✓	✓	-	✓	✓
Luận văn	DSL đặc tả RBAC trong DDD	✓	✓	✓	✓	✓

(1) Đặc tả ở mức mô hình (2) Liên kết với mô hình miền (3) Phân cấp vai trò và ràng buộc RBAC (4) Ngữ nghĩa hình thức kiểm chứng được (5) Sinh mã tự động

1.4 Mục tiêu và câu hỏi nghiên cứu

1.4.1 Phát biểu bài toán

Bài toán nghiên cứu được phát biểu theo dạng *Cho – Tìm – Sao cho*:

Cho:

các đặc tả ở mức mô hình của một hệ thống phần mềm, gồm (i) sơ đồ lớp mô tả mô hình miền cùng với các vai trò, quyền và phân cấp vai trò; (ii) sơ đồ hoạt động mô tả hành vi của các ca sử dụng; và (iii) tập ràng buộc hình thức đặc tả các bất biến của chính sách phân quyền.

Tìm:

một phương pháp cùng công cụ hỗ trợ, biến đổi tự động các đặc tả trên thành một ứng dụng phân tầng theo DDD kèm bộ thực thi phân quyền đúng với chính sách đã đặc tả.

Sao cho:

(a) phân quyền là khía cạnh hạng nhất của mô hình, ngang hàng với cấu trúc và hành vi; (b) tính trực giao được bảo toàn – thêm hoặc bớt khía cạnh phân quyền không phá vỡ hai khía cạnh còn lại; (c) tồn tại quan hệ truy vết hai chiều giữa mô hình và mã nguồn; (d) toàn bộ quá trình tự động và lặp lại được.

1.4.2 Câu hỏi nghiên cứu

Từ phát biểu trên, luận văn đặt ra bốn câu hỏi nghiên cứu, được trả lời bằng thực nghiệm ở Chương 4:

RQ1. Tính khả thi: phương pháp và công cụ đề xuất có sinh được một hệ thống hoạt động đúng với đặc tả RBAC đầu vào hay không?

RQ2. Năng suất: so với cài đặt thủ công, phương pháp tiết kiệm được bao nhiêu công sức đặc tả và bảo trì chính sách phân quyền?

RQ3. Tính đúng đắn: các vi phạm ràng buộc phân quyền trong đặc tả có được phát hiện và ngăn chặn trước khi sinh mã hay không?

RQ4. Truy vết và mức dễ hiểu: mã nguồn sinh ra có đối chiếu ngược được về phần tử mô hình tương ứng, và người đọc sơ đồ có nắm được chính sách phân quyền của hệ thống hay không?

1.5 Đóng góp của luận văn

Luận văn có hai đóng góp chính, có quan hệ tương hỗ: đóng góp thứ hai hiện thực hóa và kiểm chứng đóng góp thứ nhất.

Đóng góp 1 - Phương pháp đặc tả và tích hợp RBAC vào mô hình miền. Luận văn phát triển một ngôn ngữ chuyên biệt miền cho phép đặc tả chính sách RBAC ngay trong mô hình miền, được trình bày theo đúng ba thành phần của một ngôn ngữ: cú pháp trừu tượng là một metamodel RBAC; cú pháp cụ thể là một profile trên sơ đồ lớp UML/PlantUML với các stereotype vai trò, cú pháp khai báo quyền có ngữ nghĩa và các stereotype ràng buộc; ngữ nghĩa tĩnh là tập bất biến OCL. Kèm theo ngôn ngữ là cơ chế tích hợp: một metamodel hợp nhất ba khía cạnh trên cùng một cây cú pháp trừu tượng, quan hệ tham chiếu chéo giữa quyền và lớp miền, và thuật toán lan truyền quyền theo phân cấp vai trò bằng phép lặp điểm bất động.

Đóng góp 2 - Công cụ hỗ trợ và đánh giá thực nghiệm. Luận văn xây dựng công cụ ddd-codegen hiện thực hóa phương pháp theo một quy trình năm bước, và đánh giá nó trên tình huống nghiên cứu CourseMan theo bốn câu hỏi nghiên cứu nêu trên, kèm phân tích các nguy cơ ảnh hưởng tính hợp lệ của kết quả.

1.6 Phạm vi và giới hạn

Để bảo đảm chiều sâu trong khuôn khổ một luận văn thạc sĩ, phạm vi nghiên cứu được giới hạn như sau:

- **Về mô hình phân quyền:** hỗ trợ đầy đủ $RBAC_0$ và $RBAC_1$ (bao gồm lan truyền quyền qua phân cấp vai trò); $RBAC_2$ được hỗ trợ ở mức đặc tả và kiểm chứng thiết kế. $RBAC_3$ và phân quyền liên miền giữa các ngữ cảnh giới hạn thuộc hướng phát triển.
- **Về ngôn ngữ ràng buộc:** chỉ hiện thực một tập con của OCL đủ để biểu diễn lớp bất biến $RBAC_0$ – $RBAC_2$; các biểu thức ngoài tập con được xử lý bằng công cụ kiểm chứng ngoài.
- **Về nền tảng đích:** phương pháp được trình bày ở mức khái niệm, độc lập với công nghệ cài đặt; một nền tảng cụ thể được chọn để minh chứng tính khả thi ở phần thực nghiệm.
- **Về đánh giá:** sử dụng một tình huống nghiên cứu; các giới hạn về khả năng khái quát hóa được phân tích trong phần nguy cơ ảnh hưởng tính hợp lệ.

1.7 Bố cục luận văn

Phần còn lại của luận văn được tổ chức như sau. **Chương 2** trình bày cơ sở lý thuyết: Thiết kế hướng miễn, ngôn ngữ chuyên biệt miễn, mô hình RBAC, các chuẩn mô hình hóa UML/OCL, cơ chế hợp nhất khía cạnh quan tâm và kiến trúc đích. **Chương 3** trình bày phương pháp đề xuất: ngôn ngữ đặc tả chính sách RBAC theo ba thành phần cú pháp trừu tượng, cú pháp cụ thể, ngữ nghĩa tĩnh; và quy trình năm bước biến đổi từ đặc tả tới mã nguồn, kèm phân tích các tính chất của phương pháp. **Chương 4** trình bày công cụ hỗ trợ, tình huống nghiên cứu, thiết kế đánh giá, kết quả thực nghiệm theo bốn câu hỏi nghiên cứu, các nguy cơ ảnh hưởng tính hợp lệ và bàn luận. Phần **Kết luận** tổng kết đóng góp, nêu hạn chế và hướng phát triển.

Chương 2

CƠ SỞ LÝ THUYẾT

Chương này trình bày các nền tảng lý thuyết mà luận văn kế thừa, được tổ chức theo bốn khối: Thiết kế hướng miền (mục 2.1), ngôn ngữ chuyên biệt miền (mục 2.2), kiểm soát truy cập dựa trên vai trò (mục 2.3) và các chuẩn mô hình hóa hình thức (mục 2.4). Mục 2.5 giới thiệu cơ chế hợp nhất khía cạnh quan tâm và kiến trúc đích. Mục 2.6 tổng kết và định vị luận văn trong bức tranh chung. Nguyên tắc lựa chọn nội dung: chỉ trình bày những khái niệm được sử dụng trực tiếp ở Chương 3 và Chương 4.

2.1 Thiết kế hướng miền

2.1.1 Ngôn ngữ chung và ngữ cảnh giới hạn

Thiết kế hướng miền (Evans, 2003) xuất phát từ một quan sát: phần lớn độ phức tạp của phần mềm nghiệp vụ không nằm ở kỹ thuật mà ở bản thân lĩnh vực nghiệp vụ. Do đó, giá trị cốt lõi của thiết kế là xây dựng được một **mô hình miền** diễn đạt trung thực tri thức nghiệp vụ và được cả chuyên gia lẫn lập trình viên cùng hiểu.

Hai khái niệm chiến lược quan trọng nhất là:

- **Ngôn ngữ chung** (*ubiquitous language*): tập thuật ngữ thống nhất, dùng đồng thời trong trao đổi, trong tài liệu và trong mã nguồn. Khi một khái niệm nghiệp vụ đổi tên, tên lớp trong mã cũng phải đổi theo.
- **Ngữ cảnh giới hạn** (*bounded context*): ranh giới trong đó một mô hình miền và ngôn ngữ chung của nó có hiệu lực. Cùng một từ có thể mang nghĩa khác nhau ở hai ngữ cảnh khác nhau; ranh giới rõ ràng cho phép mỗi mô hình giữ được tính nhất quán nội tại.

Ngữ cảnh giới hạn có ý nghĩa đặc biệt đối với luận văn: nó vừa là đơn vị phân rã của mô hình miền, vừa là đơn vị phân rã của kiến trúc đích, và cũng chính là phạm vi hiệu lực tự nhiên của một tập vai trò trong chính sách phân quyền.

2.1.2 Các khuôn mẫu chiến thuật

Ở mức chiến thuật, DDD cung cấp một bộ khuôn mẫu để cấu trúc mô hình miền (Evans, 2003; Vernon, 2013). Bảng 2.1 tóm tắt các khuôn mẫu được luận văn sử dụng.

Bảng 2.1: Các khuôn mẫu chiến thuật của DDD được sử dụng trong luận văn

Khuôn mẫu	Ý nghĩa
Thực thể (<i>entity</i>)	Đối tượng có định danh riêng, tồn tại xuyên suốt vòng đời dù thuộc tính thay đổi.
Đối tượng giá trị (<i>value object</i>)	Đối tượng bất biến, được xác định bởi giá trị chứ không bởi định danh.
Gốc tụ hợp (<i>aggregate root</i>)	Thực thể đóng vai trò cửa ngõ của một tụ hợp, bảo đảm bất biến nghiệp vụ của cả tụ hợp; là đơn vị nhất quán của giao dịch.
Kho lưu trữ (<i>repository</i>)	Giao diện truy xuất tụ hợp, tách mô hình miền khỏi cơ chế lưu trữ cụ thể.
Dịch vụ miền (<i>domain service</i>)	Thao tác nghiệp vụ không thuộc riêng một thực thể nào, thường trải rộng trên nhiều tụ hợp.
Sự kiện miền (<i>domain event</i>)	Sự kiện có ý nghĩa nghiệp vụ đã xảy ra, dùng để truyền tin giữa các phần của hệ thống.

Sáu khuôn mẫu này, cùng với khái niệm ca sử dụng ở tầng ứng dụng và khái niệm vai trò ở khía cạnh phân quyền, tạo thành tập tám khái niệm mà ngôn ngữ đặc tả của luận văn cần biểu diễn được (chi tiết ở Chương 3).

2.1.3 Kiến trúc phân tầng

Để giữ mô hình miền độc lập với hạ tầng, DDD khuyến nghị tổ chức mã nguồn theo bốn tầng (Vernon, 2013):

- **Tầng miền** chứa các lớp miền và quy tắc nghiệp vụ thuần túy, không phụ thuộc bất kỳ khung phần mềm nào.
- **Tầng ứng dụng** điều phối các ca sử dụng, gọi tới tầng miền và tầng hạ tầng nhưng không chứa quy tắc nghiệp vụ.
- **Tầng hạ tầng** hiện thực hóa các giao diện kỹ thuật: lưu trữ, gửi thông điệp, xác thực.
- **Tầng trình bày** tiếp nhận yêu cầu từ bên ngoài và trả kết quả.

Vị trí của kiểm soát truy cập trong kiến trúc này là một quyết định thiết kế đáng lưu ý: điểm kiểm soát nên đặt ở tầng trình bày và tầng ứng dụng, còn tầng miền phải được giữ “sạch” – đây chính là điều kiện bảo đảm tính trực giao giữa khía cạnh phân quyền và khía cạnh cấu trúc.

2.2 Ngôn ngữ chuyên biệt miền

2.2.1 Khái niệm và phân loại

Ngôn ngữ chuyên biệt miền (*domain-specific language* - DSL) là ngôn ngữ máy tính có sức biểu đạt hạn chế, tập trung vào một lĩnh vực cụ thể (Fowler, 2010). Đối lại sự hạn chế đó, DSL cho phép diễn đạt bài toán bằng chính khái niệm của lĩnh vực, ngắn gọn và dễ kiểm chứng hơn so với ngôn ngữ lập trình đa dụng.

Fowler (2010) phân biệt hai loại: **DSL nội sinh** (*internal*) được nhúng trong cú pháp của một ngôn ngữ chủ, và **DSL ngoại sinh** (*external*) có cú pháp riêng cùng bộ phân tích riêng. Một khái niệm then chốt khác của Fowler là **mô hình ngữ nghĩa** (*semantic model*): cấu trúc dữ liệu trung gian mà bộ phân tích tạo ra và bộ sinh mã tiêu thụ. Việc tách bạch mô hình ngữ nghĩa khỏi cú pháp cho phép thay đổi cú pháp mà không ảnh hưởng phần sinh mã, và ngược lại.

2.2.2 Ba thành phần của một ngôn ngữ

Voelter et al. (2013) hệ thống hóa quy trình xây dựng DSL quanh ba thành phần, và đây cũng là khung trình bày mà Chương 3 của luận văn tuân theo:

1. **Cú pháp trừu tượng** (*abstract syntax*): tập khái niệm của ngôn ngữ cùng quan hệ giữa chúng, thường đặc tả bằng một metamodel. Đây là “xương sống” của ngôn ngữ, độc lập với cách viết.
2. **Cú pháp cụ thể** (*concrete syntax*): ký pháp mà người dùng thực sự viết hoặc vẽ. Một cú pháp trừu tượng có thể có nhiều cú pháp cụ thể (văn bản, đồ họa, dạng bảng).
3. **Ngữ nghĩa** (*semantics*): ý nghĩa của các cấu trúc ngôn ngữ. Trong thực hành thường tách thành *ngữ nghĩa tĩnh* – các ràng buộc mà một mô hình hợp lệ phải thỏa mãn, và *ngữ nghĩa động* – hành vi khi thực thi, thường được định nghĩa gián tiếp qua phép biến đổi sang một ngôn ngữ đích.

Nhận xét 2.1. Cách phân chia này giải thích vì sao một DSL không đồng nghĩa với “một bộ sinh mã”. Bộ sinh mã chỉ hiện thực hóa ngữ nghĩa động; phần giá trị lâu dài của một DSL nằm ở cú pháp trừu tượng và ngữ nghĩa tĩnh – những thứ độc lập với công nghệ đích và có thể tái sử dụng khi nền tảng thay đổi.

2.2.3 Ngôn ngữ chuyên biệt miền dựa trên chú giải cho DDD

Hướng tiếp cận **aDSL** (Le et al., 2018) chọn cú pháp cụ thể là các chú giải (*annotation*) gắn trực tiếp lên phần tử của ngôn ngữ lập trình chủ. Ưu điểm là đặc tả nằm ngay cạnh mã nguồn, dùng lại được hệ công cụ sẵn có của ngôn ngữ chủ; đặc tả và mã không bao giờ nằm ở hai nơi tách rời.

DCSL (Le et al., 2018, 2020) là aDSL cho khía cạnh cấu trúc, với các khái niệm chính:

- **DClass** – lớp miền, có thuộc tính mutable phân biệt lớp khả biến với lớp bất biến;
- **DAttr** – thuộc tính miền, mang các ràng buộc như bắt buộc, độ dài, khóa định danh;

- DAssoc – liên kết giữa các lớp miền, mang thông tin về loại liên kết và bội số.

AGL (Dang et al., 2023; Le et al., 2023) bổ sung khía cạnh hành vi bằng khái niệm *đồ thị hoạt động*: mỗi ca sử dụng được mô tả bởi một chuỗi hành động có thứ tự (ANode, MAct), lấy từ sơ đồ hoạt động UML. Điểm quan trọng về mặt phương pháp là AGL không thay thế DCSL mà *bổ sung* vào cùng một mô hình miền – tiền đề trực tiếp cho ý tưởng hợp nhất nhiều khía cạnh.

Nhận xét 2.2. Luận văn kế thừa tinh thần aDSL nhưng chọn cú pháp cụ thể là *sơ đồ lớp dạng văn bản* thay vì chú giải trên mã nguồn. Lý do: chính sách phân quyền là đối tượng cần được chuyên gia nghiệp vụ và cán bộ quản trị xem xét, nên một ký pháp trực quan có lợi thế hơn chú giải trong mã. Ánh xạ giữa chú giải trong ngôn ngữ chủ và stereotype trên sơ đồ là ánh xạ một-một, nên tinh thần aDSL được bảo toàn.

2.3 Kiểm soát truy cập dựa trên vai trò

2.3.1 Mô hình cơ sở RBAC₀

Sandhu et al. (1996) định nghĩa họ mô hình RBAC gồm bốn cấp. Mô hình cơ sở RBAC₀ được phát biểu như sau.

Định nghĩa 2.1 (RBAC₀). Mô hình RBAC₀ gồm các tập hợp U (người dùng), R (vai trò), P (quyền), S (phiên làm việc) cùng các quan hệ:

$UA \subseteq U \times R$	(gán vai trò cho người dùng)
$PA \subseteq P \times R$	(gán quyền cho vai trò)
$user : S \rightarrow U$	(chủ thể của phiên)
$roles : S \rightarrow 2^R$	(các vai trò được kích hoạt trong phiên)

với ràng buộc $roles(s) \subseteq \{r \mid (user(s), r) \in UA\}$: người dùng chỉ có thể kích hoạt những vai trò đã được gán cho mình.

Điểm cốt lõi của RBAC₀ là *lớp trung gian vai trò*: quyền không gán trực tiếp cho người dùng. Nhờ đó, thay đổi nhân sự chỉ ảnh hưởng quan hệ UA , còn thay đổi chính sách chỉ ảnh hưởng quan hệ PA – hai loại thay đổi được tách bạch.

Trong mô hình gốc, quyền được xem là một phần tử trừu tượng của tập P . Luận văn tinh chỉnh khái niệm này bằng cách gán cho quyền một cấu trúc nội tại gồm tài nguyên, thao tác và phạm vi, trong đó tài nguyên là một lớp của mô hình miền – đây chính là điểm nối giữa khía cạnh phân quyền và khía cạnh cấu trúc (Chương 3).

2.3.2 RBAC₁: phân cấp vai trò

Định nghĩa 2.2 (RBAC₁). RBAC₁ mở rộng RBAC₀ bằng quan hệ thứ tự bộ phận $RH \subseteq R \times R$ trên tập vai trò. Ký hiệu $r_1 \succeq r_2$ nếu r_1 kế thừa r_2 . Khi đó tập quyền hiệu dụng của vai trò r là

$$perm^*(r) = \bigcup_{r' \preceq r} \{p \mid (p, r') \in PA\}.$$

Nói cách khác, vai trò cấp cao thừa hưởng toàn bộ quyền của các vai trò cấp thấp hơn nó trong phân cấp. Quan hệ RH là thứ tự bộ phận nên phải phản xạ, bắc cầu và phản đối xứng; hệ quả thực tiễn quan trọng là *đồ thị kế thừa vai trò không được chứa chu trình*. Bất biến này được luận văn kiểm tra tường minh ở bước kiểm chứng, vì một chu trình sẽ khiến phép tính $perm^*$ mất ý nghĩa thứ tự và làm chính sách trở nên vô nghĩa (mọi vai trò trong chu trình có quyền như nhau).

2.3.3 RBAC₂: ràng buộc

RBAC₂ bổ sung các ràng buộc trên những quan hệ đã có. Ba nhóm ràng buộc được luận văn hỗ trợ:

Tách nhiệm tĩnh (SSD).

Hai vai trò xung khắc không được gán đồng thời cho cùng một người dùng:

$$\forall u \in U : \neg((u, r_1) \in UA \wedge (u, r_2) \in UA) \quad \text{với } (r_1, r_2) \in SSD.$$

Ràng buộc này quyết định được ngay ở mức mô hình vì chỉ phụ thuộc quan hệ UA .

Tách nhiệm động (DSD).

Hai vai trò xung khắc có thể cùng được gán nhưng không được kích hoạt trong cùng một phiên:

$$\forall s \in S : \neg(r_1 \in roles(s) \wedge r_2 \in roles(s)) \quad \text{với } (r_1, r_2) \in DSD.$$

Vì phụ thuộc trạng thái phiên nên ràng buộc này chỉ kiểm tra được lúc vận hành.

Giới hạn số lượng (cardinality).

Số người dùng được gán một vai trò không vượt quá ngưỡng cho trước: $|\{u \mid (u, r) \in UA\}| \leq max(r)$.

Nhận xét 2.3. Sự khác biệt giữa SSD và DSD minh họa một nguyên tắc thiết kế xuyên suốt luận văn: *ràng buộc nào quyết định được từ mô hình thì kiểm chứng ở thời điểm thiết kế; ràng buộc nào phụ thuộc trạng thái vận hành thì phải sinh thành điểm kiểm tra trong mã*. Nguyên tắc này được cụ thể hóa ở bước kiểm chứng của phương pháp (Chương 3).

RBAC₃ hợp nhất phân cấp vai trò với ràng buộc, làm nảy sinh các tương tác tinh tế – chẳng hạn hai vai trò xung khắc có thể cùng bị kế thừa bởi một vai trò cấp trên. Luận văn không xử lý cấp này và xếp vào hướng phát triển.

2.3.4 Biểu diễn RBAC bằng UML và OCL

Ray et al. (2004) đề xuất biểu diễn chính sách RBAC bằng UML kết hợp OCL: các khái niệm vai trò, quyền, phiên được mô hình hóa bằng lớp UML, còn các ràng buộc RBAC₂ được phát biểu bằng bất biến OCL. Đóng góp đáng chú ý của công trình là ý tưởng *mẫu vi phạm*: mỗi bất biến đi kèm một mẫu sơ đồ đối tượng mô tả tình huống vi phạm, giúp trực quan hóa lỗi cho người thiết kế. Luận văn kế thừa cả cách phát biểu bất biến lẫn nguyên tắc chẩn đoán này, với một khác biệt về phạm vi: thay vì mô hình hóa RBAC như

hệ thống độc lập, luận văn nhúng nó vào mô hình miền của ứng dụng thông qua quan hệ tham chiếu chéo.

Basin et al. (2006) đi xa hơn về phía sinh mã với SecureUML: chính sách được đặc tả bằng một UML profile và hạ tầng kiểm soát truy cập được sinh tự động. Đây là công trình gần nhất với luận văn; điểm khác biệt đã được phân tích ở mục 1.3.

2.3.5 Phân quyền đa miền

Khi hệ thống được phân rã thành nhiều ngữ cảnh giới hạn hoặc nhiều tổ chức hợp tác, xuất hiện bài toán phân quyền liên miền. Li et al. (2022) đề xuất mô hình RBAC cho môi trường đa miền, với các khái niệm vai trò trừu tượng và vai trò cụ thể, cơ chế cô lập giữa các miền và ánh xạ vai trò liên miền có kiểm soát. Ba lớp bất biến mới nảy sinh trong bối cảnh này – chống leo thang đặc quyền, đồ thị ánh xạ vai trò không chu trình, và tách nhiệm xuyên miền – là cơ sở lý thuyết cho hướng mở rộng của luận văn.

2.4 Chuẩn mô hình hóa hình thức và công cụ

2.4.1 Cơ chế profile của UML

UML (omg, 2017) cung cấp cơ chế mở rộng nhẹ gọi là **profile**: thay vì sửa đổi metamodel chuẩn, người thiết kế định nghĩa các *stereotype* gắn lên phần tử mô hình để bổ sung ngữ nghĩa chuyên biệt cho một lĩnh vực. Đây là cách chuẩn hóa để một cộng đồng định nghĩa “phương ngữ” UML riêng mà vẫn dùng được công cụ UML thông thường. Luận văn định nghĩa cú pháp cụ thể của ngôn ngữ đặc tả RBAC theo đúng cơ chế này.

2.4.2 Ngôn ngữ ràng buộc đối tượng

OCL (omg, 2014) là ngôn ngữ khai báo, không có tác dụng phụ, dùng để phát biểu ràng buộc trên mô hình UML. Các cấu trúc được luận văn sử dụng gồm: khai báo bất biến trong một ngữ cảnh (*context...inv*), các phép trên tập hợp (*forAll*, *exists*, *includes*, *isUnique*), phép duyệt quan hệ và các biểu thức lô-gic. Ví dụ 2.1 minh họa hai bất biến RBAC viết bằng OCL.

Ví dụ 2.1. Bất biến về tính duy nhất của tên vai trò và bất biến chỉ được kích hoạt vai trò đã được gán:

```
context Role
  inv UniqueRoleName:
    Role.allInstances()->isUnique(name)

context User
  inv OnlyAssignedRolesActivated:
    self.sessions->forAll(s |
      s.activeRoles->forAll(r | self.assignedRoles->includes(r)))
```

Listing 2.1: Hai bất biến RBAC viết bằng OCL

2.4.3 Công cụ kiểm chứng USE

USE (Gogolla et al., 2007) là môi trường đặc tả và kiểm chứng mô hình UML/OCL: người dùng khai báo mô hình cùng các bất biến, tạo các ảnh chụp trạng thái (*snapshot*) dưới dạng sơ đồ đối tượng, và công cụ báo bất biến nào bị vi phạm. USE được luận văn xác định là công cụ cho bước kiểm chứng ở mức thiết kế trong hướng hoàn thiện.

2.4.4 PlantUML như một cú pháp văn bản của sơ đồ UML

PlantUML (pla, 2025) cho phép mô tả sơ đồ UML bằng văn bản thuần và kết xuất thành hình vẽ. So với các công cụ mô hình hóa đồ họa, ký pháp văn bản có ba ưu điểm quan trọng đối với luận văn: quản lý được bằng hệ thống quản lý phiên bản như mã nguồn; so sánh khác biệt giữa hai phiên bản dễ dàng; và tự động hóa được trong quy trình tích hợp liên tục. Hạn chế của PlantUML là không có metamodel hình thức đi kèm – đây chính là lý do luận văn phải tự định nghĩa cú pháp trừu tượng dưới dạng metamodel riêng ở Chương 3.

2.5 Hợp nhất khía cạnh quan tâm và kiến trúc đích

2.5.1 Cơ chế hợp nhất của UDML

UDML (Le and Dang, 2025) giải quyết bài toán: làm thế nào để nhiều khía cạnh quan tâm cùng tồn tại trong một mô hình miền thống nhất mà không cản trở nhau. Giải pháp dựa trên hai khái niệm: *Annotable* – phần tử mô hình có thể được gắn chú giải, và *Annotation* – đơn vị thông tin thuộc về một khía cạnh, gắn vào một phần tử và giữ tham chiếu ngược tới phần tử đó. Nhờ vậy, các khía cạnh có thể tham chiếu chéo lẫn nhau ngay trong cây cú pháp trừu tượng.

Ba tính chất mà một phép hợp nhất đúng đắn phải bảo đảm, theo UDML, là:

- **Tính trực giao:** thêm hoặc bớt một khía cạnh không làm thay đổi ngữ nghĩa của các khía cạnh còn lại.
- **Tính gắn kết:** mọi tham chiếu chéo giữa các khía cạnh đều phân giải được về một phần tử tồn tại.
- **Tính truy vết:** mỗi phần tử của mô hình hợp nhất giữ được liên kết về vị trí xuất xứ trong đặc tả nguồn.

Ba tính chất này được luận văn sử dụng làm tiêu chí thiết kế cho bước hợp nhất mô hình và được kiểm tra lại trong phần đánh giá.

2.5.2 Kiến trúc vi dịch vụ phân tầng TMSA

TMSA (Le et al., 2025) đề xuất tổ chức các dịch vụ của một hệ thống theo cấu trúc đa cây: mỗi dịch vụ là một cây các mô-đun hướng miền, các cạnh liên cây cho phép tái sử dụng cấu trúc giữa các dịch vụ. Metamodel của TMSA được đặc tả bằng UML/OCL, cho phép định nghĩa hình thức các mẫu chịu lỗi và các phép tiến hóa kiến trúc. Đối với luận

văn, TMSA cung cấp định hướng về cấu trúc đầu ra: mỗi ngữ cảnh giới hạn tương ứng một mô-đun dịch vụ phân tầng, và điểm kiểm soát truy cập được đặt tại ranh giới của mô-đun đó.

2.5.3 Nền tảng cài đặt NestJS

NestJS (nes, 2025) là khung phát triển ứng dụng phía máy chủ trên nền Node.js với kiến trúc mô-đun rõ ràng. Ba cơ chế của nền tảng này được bước sinh mã sử dụng trực tiếp:

- **Guard** – thành phần chặn yêu cầu trước khi tới bộ xử lý, nơi tự nhiên để đặt điểm kiểm soát truy cập;
- **Custom decorator và metadata reflection** – cơ chế gắn siêu dữ liệu (vai trò, quyền yêu cầu) lên từng điểm cuối và đọc lại lúc chạy;
- **Mô-đun CQRS** – tách lệnh khỏi truy vấn, phù hợp với cách AGL mô tả ca sử dụng như một chuỗi hành động.

Việc chọn TypeScript làm ngôn ngữ đích cũng có lý do về mặt phương pháp: *decorator* của TypeScript là tương đương trực tiếp của *annotation* trong Java, nên tinh thần aDSL của mạch DCSL/AGL được bảo toàn nguyên vẹn khi chuyển sang hệ sinh thái này.

2.6 Tổng kết chương

Chương này đã trình bày năm khối kiến thức mà luận văn kế thừa. Từ DDD, luận văn lấy khái niệm mô hình miền, ngữ cảnh giới hạn, các khuôn mẫu chiến thuật và kiến trúc phân tầng. Từ lý thuyết DSL, luận văn lấy khung ba thành phần – cú pháp trừu tượng, cú pháp cụ thể, ngữ nghĩa – làm bố cục trình bày cho ngôn ngữ đề xuất, cùng tinh thần aDSL của DCSL và AGL. Từ RBAC, luận văn lấy ngữ nghĩa hình thức của các cấp $RBAC_0$ – $RBAC_2$ (Định nghĩa 2.1 và 2.2) làm cơ sở cho metamodel phân quyền và cho thuật toán lan truyền quyền. Từ UML/OCL, luận văn lấy cơ chế profile cho cú pháp cụ thể và ngôn ngữ OCL cho ngữ nghĩa tĩnh. Từ UDML và TMSA, luận văn lấy cơ chế hợp nhất khía cạnh cùng ba tính chất trực giao, gắn kết, truy vết, và định hướng cấu trúc đầu ra.

Điểm mà các công trình trên còn để ngỏ – và cũng là nội dung của chương tiếp theo – là một ngôn ngữ đặc tả chính sách RBAC nằm ngay trong mô hình miền, có ngữ nghĩa hình thức kiểm chứng được, kèm cơ chế biến đổi tự động từ đặc tả đó tới phần mềm thực thi.

Chương 3

PHƯƠNG PHÁP ĐỀ XUẤT

Chương này trình bày đóng góp chính của luận văn. Mục 3.1 phác thảo toàn cảnh phương pháp ở mức khái niệm. Mục 3.2 định nghĩa ngôn ngữ đặc tả chính sách RBAC theo ba thành phần đã nêu ở mục 2.2.2. Năm mục tiếp theo (3.3–3.7) trình bày chi tiết từng bước của quy trình biến đổi. Mục 3.8 phân tích các tính chất mà phương pháp bảo đảm.

3.1 Tổng quan phương pháp

3.1.1 Nguyên tắc thiết kế

Phương pháp được xây dựng trên bốn nguyên tắc, mỗi nguyên tắc trả lời trực tiếp một yêu cầu đã nêu trong phát biểu bài toán ở mục 1.4:

1. **Một nguồn sự thật duy nhất.** Chính sách phân quyền chỉ được khai báo ở một nơi – mô hình – và mọi biểu hiện khác của nó (mã nguồn, cấu hình, tài liệu) đều được sinh ra từ đó. Nguyên tắc này loại trừ khả năng mô hình và mã phân kỳ.
2. **Phân quyền là khía cạnh hạng nhất.** Vai trò, quyền và ràng buộc là các phần tử của mô hình miền, có metamodel riêng và tham chiếu trực tiếp tới lớp miền, chứ không phải siêu dữ liệu phụ trợ gắn ngoài.
3. **Kiểm chứng trước khi sinh.** Mô hình phải vượt qua bước kiểm chứng bất biến thì mới được chuyển sang bước sinh mã. Một đặc tả sai không bao giờ tạo ra mã.
4. **Độc lập với công nghệ cài đặt.** Bốn bước đầu chỉ thao tác trên các khái niệm mô hình hóa; mọi tri thức về nền tảng đích được cô lập trong bộ mẫu sinh của bước cuối.

3.1.2 Kiến trúc tổng thể

Phương pháp có dạng một quy trình biến đổi một chiều gồm năm bước, chuyển đặc tả ở mức mô hình thành phần mềm thực thi. Hình 3.1 minh họa đầu vào, các bước xử lý, dữ liệu trung gian và đầu ra.

Đầu vào gồm ba tài liệu đặc tả bổ sung cho nhau:

[Hình 3.1 – Sơ đồ tổng quan phương pháp: ba tài liệu đặc tả đầu vào → năm bước xử lý → bốn sản phẩm dữ liệu trung gian → ứng dụng phân tầng kèm bộ thực thi phân quyền, kèm vòng phản hồi từ bước kiểm chứng về đặc tả]

Hình 3.1: Tổng quan phương pháp ở mức khái niệm

- *Sơ đồ lớp* mô tả đồng thời cấu trúc miền (các lớp, thuộc tính, quan hệ) và chính sách phân quyền (vai trò, quyền, phân cấp, ràng buộc). Việc đặt hai khía cạnh trên cùng một sơ đồ là chủ ý: nó cho phép quyền tham chiếu trực tiếp tới lớp miền mà không cần cơ chế liên kết ngoài.
- *Sơ đồ hoạt động* mô tả hành vi của từng ca sử dụng dưới dạng chuỗi hành động, theo ngôn ngữ AGL.
- *Tập ràng buộc* chứa các bất biến OCL – phần ngữ nghĩa hình thức mà ký pháp sơ đồ không diễn đạt hết.

Năm bước xử lý lần lượt là: phân tích cú pháp, hợp nhất mô hình, làm giàu mô hình, kiểm chứng ràng buộc và sinh mã theo mẫu.

Bốn sản phẩm dữ liệu trung gian nằm giữa các bước: cây cú pháp của từng nguồn, mô hình miền hợp nhất, mô hình đã làm giàu, và mô hình hợp lệ. Việc đặt tên và định nghĩa rõ từng sản phẩm trung gian không chỉ nhằm trình bày: nó cho phép mỗi bước được kiểm thử độc lập, và cho phép chèn thêm bước mới trong tương lai mà không phá vỡ các bước hiện có.

Vòng phản hồi đi từ bước kiểm chứng ngược về đặc tả: khi phát hiện vi phạm, quy trình dừng lại và trả về chẩn đoán trở tới vị trí gây lỗi trong tài liệu đặc tả gốc, người thiết kế hiệu chỉnh rồi chạy lại. Đây là điểm khác biệt cơ bản so với các bộ sinh mã thông thường, vốn chỉ kiểm tra tính hợp lệ cú pháp chứ không kiểm chứng ngữ nghĩa của chính sách.

Nhận xét 3.1. Toàn bộ nội dung mục 3.1 không nhắc tới bất kỳ ngôn ngữ lập trình hay khung phần mềm nào. Đây là chủ ý: phương pháp được phát biểu ở mức khái niệm, và việc chọn một nền tảng đích cụ thể chỉ diễn ra ở bước 5 và ở phần thực nghiệm.

3.2 Ngôn ngữ đặc tả chính sách RBAC

Mục này trình bày đóng góp thứ nhất của luận văn theo đúng ba thành phần của một ngôn ngữ: cú pháp trừu tượng, cú pháp cụ thể và ngữ nghĩa tĩnh.

3.2.1 Cú pháp trừu tượng: metamodel RBAC

Cú pháp trừu tượng được đặc tả bằng một metamodel gồm bảy khái niệm. Định nghĩa 3.1 phát biểu hình thức cấu trúc này.

Định nghĩa 3.1 (Metamodel chính sách RBAC). Một chính sách phân quyền là bộ

$$\mathcal{P} = \langle \mathcal{R}, \mathcal{Q}, \text{grant}, \text{extends}, \mathcal{C}_{ssd}, \mathcal{C}_{dsd}, \text{card} \rangle$$

trong đó:

- $\mathcal{R}$ là tập hữu hạn các *vai trò*, mỗi vai trò có tên duy nhất;
- $\mathcal{Q}$ là tập hữu hạn các *quyền*; mỗi quyền $q \in \mathcal{Q}$ là bộ ba $\langle \text{res}(q), \text{act}(q), \text{scope}(q) \rangle$ với $\text{res}(q)$ là một lớp của mô hình miền, $\text{act}(q)$ là một thao tác và $\text{scope}(q)$ là phạm vi áp dụng;
- $\text{grant} \subseteq \mathcal{R} \times \mathcal{Q}$ là quan hệ gán quyền cho vai trò (tương ứng quan hệ PA trong Định nghĩa 2.1);
- $\text{extends} \subseteq \mathcal{R} \times \mathcal{R}$ là quan hệ kế thừa vai trò (tương ứng RH trong Định nghĩa 2.2);
- $\mathcal{C}_{ssd}, \mathcal{C}_{dsd} \subseteq \mathcal{R} \times \mathcal{R}$ là các tập cặp vai trò xung khắc tĩnh và động;
- $\text{card} : \mathcal{R} \rightarrow \mathbb{N}$ là hàm bộ phận gán giới hạn số người giữ vai trò.

Điểm cần nhấn mạnh trong Định nghĩa 3.1 là thành phần $\text{res}(q)$: nó không phải một chuỗi ký tự tự do mà là *tham chiếu tới một lớp của mô hình miền*. Chính thành phần này biến chính sách phân quyền từ một danh sách nhãn rời rạc thành một phần gắn kết của mô hình, và cũng chính nó đặt ra yêu cầu về tính gắn kết sẽ được kiểm tra ở bước 4.

[Hình 3.2 – Metamodel chính sách RBAC dạng sơ đồ lớp: Role, Permission, Resource, Action, Scope, RoleHierarchy, SsdConstraint, DsdConstraint, Cardinality và quan hệ giữa chúng]

Hình 3.2: Metamodel chính sách RBAC (cú pháp trừu tượng)

3.2.2 Cú pháp cụ thể: profile trên sơ đồ lớp

Cú pháp cụ thể được định nghĩa dưới dạng một UML profile theo cơ chế stereotype đã trình bày ở mục 2.4. Bảng 3.1 là ánh xạ đầy đủ giữa khái niệm metamodel và ký pháp trên sơ đồ.

Cấu trúc nội tại của biểu thức quyền được đặc tả bằng văn phạm sau:

```
Permission ::= Name ":" Resource "." Action [ "[" Scope "]" ]
Name        ::= UPPER_IDENT
Resource    ::= IDENT          (* ten mot lop mien tren cung so do *)
Action      ::= "create" | "read" | "update" | "delete" | IDENT
Scope       ::= "all" | "own" | IDENT
```

Listing 3.1: Văn phạm của biểu thức quyền (ký pháp EBNF)

Ba quyết định thiết kế trong văn phạm này cần được lý giải:

Bảng 3.1: Ánh xạ giữa cú pháp trừu tượng và cú pháp cụ thể

Khái niệm	Ký pháp trên sơ đồ	Ghi chú
Vai trò	<code>class R <<Role>></code>	Tên lớp là tên vai trò
Quyền	Trường trong thân lớp vai trò, dạng <code>TEN : TaiNguyen.thaoTac [phamVi]</code>	Xem văn phạm 3.1
Gán quyền	Quyền được khai báo bên trong lớp vai trò	Quan hệ <i>grant</i> suy ra từ vị trí khai báo
Kế thừa vai trò	<code>R1 -- > R2</code>	R_1 kế thừa R_2
Tách nhiệm tĩnh	<code>R1 ..> R2 : <<SSD>></code>	Cặp không đối xứng về ký pháp, đối xứng về ngữ nghĩa
Tách nhiệm động	<code>R1 ..> R2 : <<DSD(maxActiveRoles=n)>></code>	
Giới hạn số lượng	<code><<cardinality max=n>></code> trong thân lớp vai trò	
Nhóm chính sách	<code>package "X" <<AccessControl>></code>	Tách chính sách khỏi phần cấu trúc trên cùng sơ đồ

- **Vì sao tách Resource và Action thay vì dùng một tên quyền tự do?** Vì chỉ khi tách bạch, hệ thống mới kiểm tra được rằng tài nguyên được nhắc tới thực sự tồn tại trong mô hình miền, và mới ánh xạ được tự động từ quyền sang các điểm cuối thao tác trên tài nguyên đó ở bước sinh mã.
- **Vì sao Scope là tùy chọn?** Để giữ ngưỡng sử dụng thấp: phần lớn quyền không cần phạm vi, và người thiết kế chỉ khai báo phạm vi khi nó thực sự tạo khác biệt (ví dụ phân biệt *đọc mọi hồ sơ* với *đọc hồ sơ của chính mình*).
- **Vì sao Action cho phép cả định danh tự do lẫn bốn thao tác chuẩn?** Bốn thao tác CRUD phủ phần lớn nhu cầu và ánh xạ được tự động sang các thao tác mặc định sinh ra ở bước 3; định danh tự do dành cho các thao tác nghiệp vụ đặc thù, khi đó tên thao tác phải trùng với tên một phương thức hoặc ca sử dụng đã khai báo.

Nhận xét 3.2. Việc chọn sơ đồ lớp làm nơi khai báo chính sách – thay vì một tệp cấu hình riêng – có một hệ quả đáng giá: chính sách phân quyền trở thành đối tượng có thể *xem xét bằng mắt* cùng lúc với mô hình miền. Chuyên gia nghiệp vụ nhìn vào một sơ đồ duy nhất là biết vai trò nào chạm được vào khái niệm nghiệp vụ nào.

3.2.3 Ngữ nghĩa tĩnh: tập bất biến

Không phải mọi mô hình viết đúng cú pháp đều là một chính sách hợp lệ. Ngữ nghĩa tĩnh của ngôn ngữ được đặc tả bằng tập bất biến trong Bảng 3.2, phân thành ba lớp theo cấp của mô hình RBAC.

Mỗi bất biến được phát biểu hình thức bằng OCL. Listing 3.2 trình bày ba bất biến tiêu biểu.

```
-- Il: ten vai tro la duy nhat
context Role
  inv UniqueRoleName:
    Role.allInstances()->isUnique(name)
```

Bảng 3.2: Tập bất biến định nghĩa ngữ nghĩa tĩnh của ngôn ngữ

Mã	Lớp	Phát biểu	Thời điểm
I1	RBAC ₀	Tên vai trò là duy nhất trong toàn chính sách	Thiết kế
I2	RBAC ₀	Mọi quyền đều có tài nguyên và thao tác xác định	Thiết kế
I3	RBAC ₀	Tài nguyên của mọi quyền phải là một lớp tồn tại trong mô hình miền	Thiết kế
I4	RBAC ₀	Thao tác tự do của quyền phải trùng tên một phương thức hoặc ca sử dụng đã khai báo	Thiết kế
I5	RBAC ₁	Đồ thị kế thừa vai trò không chứa chu trình	Thiết kế
I6	RBAC ₂	Hai vai trò xung khắc tĩnh không được gán cho cùng một người dùng	Thiết kế & vận hành
I7	RBAC ₂	Hai vai trò xung khắc động không được kích hoạt trong cùng một phiên	Vận hành
I8	RBAC ₂	Số người giữ một vai trò không vượt quá giới hạn khai báo	Vận hành

```
-- I3: tai nguyen cua quyen phai la mot lop mien ton tai
context Permission
  inv ResourceExists:
    DomainClass.allInstances()->exists(c | c.name = self.resource)

-- I5: do thi ke thua vai tro khong co chu trinh
context Role
  inv AcyclicHierarchy:
    not self.allParents()->includes(self)
```

Listing 3.2: Đặc tả OCL của các bất biến I1, I3 và I5

Cột “Thời điểm” của Bảng 3.2 thể hiện nguyên tắc đã nêu ở Nhận xét 2.3: bất biến nào quyết định được từ mô hình thì kiểm ở thời điểm thiết kế; bất biến phụ thuộc trạng thái vận hành (phiên làm việc, số người dùng thực tế) buộc phải sinh thành điểm kiểm tra trong mã. Riêng I6 xuất hiện ở cả hai cột: phép gán vai trò khai báo tĩnh trong đặc tả kiểm được lúc thiết kế, còn phép gán phát sinh lúc hệ thống đang chạy phải kiểm lúc vận hành.

3.3 Bước 1 - Phân tích cú pháp

3.3.1 Mục tiêu và đầu ra

Bước 1 chuyển ba tài liệu đặc tả dạng văn bản thành cấu trúc dữ liệu mà máy xử lý được. Đầu ra là ba cây cú pháp độc lập, mỗi cây phản ánh trung thực nội dung của một nguồn, chưa có liên kết chéo giữa chúng.

Nguyên tắc thiết kế của bước này là *trung thực và không suy diễn*: bộ phân tích chỉ ghi nhận những gì được viết ra, không tự bổ sung phần tử nào. Mọi phép suy diễn được đẩy sang bước 3, nơi chúng được thực hiện tập trung và kiểm thử được.

3.3.2 Xử lý từng nguồn đặc tả

Sơ đồ lớp. Bộ phân tích duyệt tuần tự tài liệu, nhận diện lần lượt: khối gói kèm stereotype ngữ cảnh giới hạn; khai báo lớp kèm stereotype; các trường và phương thức trong thân lớp; và các quan hệ giữa lớp. Với lớp mang stereotype vai trò, các trường trong thân lớp được phân tích theo văn phạm 3.1 thành đối tượng quyền có cấu trúc thay vì chuỗi ký tự.

Sơ đồ hoạt động. Mỗi làn bơi (*swimlane*) xác định ca sử dụng hoặc lớp miền mà chuỗi hành động thuộc về; các hành động liên tiếp trong một làn được gom thành một nút hoạt động có thứ tự; cấu trúc rẽ nhánh và rẽ song song được ghi nhận thành nút quyết định tương ứng.

Tập ràng buộc. Mỗi bất biến được phân tích thành cây biểu thức có kiểu, gồm ngữ cảnh, tên bất biến và thân biểu thức.

3.3.3 Vị trí nguồn và chẩn đoán lỗi

Một quyết định thiết kế xuyên suốt: *mỗi phần tử của cây cú pháp mang theo vị trí xuất xứ* – tên tập và số dòng nơi nó được khai báo. Chi phí là một trường dữ liệu bổ sung trên mỗi nút; lợi ích thì kéo dài tới tận cuối quy trình:

- Bước 4 báo vi phạm bất biến kèm đúng dòng gây lỗi thay vì chỉ nêu tên phần tử;
- Bước 5 chèn được chú thích truy vết vào mã sinh ra, tạo nên quan hệ truy vết hai chiều;
- Người thiết kế sửa lỗi trực tiếp trên tài liệu đặc tả mà không phải dò ngược.

Về phạm vi cú pháp, phương pháp chủ trương hỗ trợ một *tập con được định nghĩa rõ* của ký pháp sơ đồ thay vì cố gắng phủ toàn bộ. Cấu trúc nằm ngoài tập con bị từ chối ngay với thông báo tường minh, thay vì bị bỏ qua âm thầm – lựa chọn này ưu tiên tính tiên đoán được của công cụ hơn là sự dễ dãi.

3.4 Bước 2 - Hợp nhất mô hình

3.4.1 Mô hình miền hợp nhất

Bước 2 gộp ba cây cú pháp thành một cấu trúc duy nhất. Định nghĩa 3.2 phát biểu cấu trúc đó.

Định nghĩa 3.2 (Mô hình miền hợp nhất). Mô hình miền hợp nhất là bộ $\mathcal{M} = \langle \mathcal{B}, \mathcal{P}, \mathcal{A}, \mathcal{I} \rangle$ trong đó $\mathcal{B}$ là tập các ngữ cảnh giới hạn (mỗi ngữ cảnh chứa các lớp miền cùng quan hệ giữa chúng – khía cạnh cấu trúc), $\mathcal{P}$ là chính sách phân quyền theo Định nghĩa 3.1 – khía cạnh phân quyền, $\mathcal{A}$ là tập các đồ thị hoạt động gắn với ca sử dụng – khía cạnh hành vi, và $\mathcal{I}$ là tập bất biến đọc từ tập ràng buộc.

Ba khía cạnh nằm trên ba nhánh riêng của cùng một cây, phản ánh trực tiếp yêu cầu về tính trực giao: có thể lược bỏ nhánh $\mathcal{P}$ mà hai nhánh còn lại vẫn là một mô hình miền hoàn chỉnh và vẫn sinh mã được – chỉ là mã không có phân quyền.

3.4.2 Tham chiếu chéo giữa các khía cạnh

Điều làm cho mô hình là *hợp nhất* chứ không phải *đặt cạnh nhau* là các tham chiếu chéo. Có ba loại:

1. **Quyền** → **lớp miền**: thành phần $res(q)$ của mỗi quyền được phân giải từ chuỗi tên thành tham chiếu trực tiếp tới đối tượng lớp miền trong nhánh B .
2. **Hành vi** → **phương thức**: mỗi nút hoạt động được gắn vào ca sử dụng hoặc phương thức tương ứng trong nhánh B .
3. **Bất biến** → **ngữ cảnh**: mỗi bất biến trong $\mathcal{I}$ được gắn vào lớp hoặc vai trò mà nó ràng buộc.

Phép phân giải tham chiếu có thể thất bại – ví dụ quyền trỏ tới một lớp không tồn tại. Khi đó bước 2 *không* báo lỗi ngay mà ghi nhận tham chiếu ở trạng thái chưa phân giải; việc phán quyết dành cho bước 4, nơi bất biến $I3$ xử lý tình huống này. Lựa chọn này giữ cho mỗi bước có một trách nhiệm duy nhất: bước 2 chỉ hợp nhất, bước 4 chỉ phán quyết.

3.4.3 Bất biến sau hợp nhất

Kế thừa ba tiêu chí của UDML (Le and Dang, 2025), mô hình sau bước 2 phải bảo đảm:

- **Trực giao**: mỗi chú giải phân quyền chỉ gắn vào loại phần tử hợp lệ (vai trò gắn ở mức lớp, không gắn ở mức thuộc tính);
- **Gắn kết**: mọi tham chiếu chéo đều phân giải được (kiểm ở bước 4);
- **Truy vết**: mọi phần tử giữ nguyên vị trí xuất xứ nhận được từ bước 1.

3.5 Bước 3 - Làm giàu mô hình

Bước 3 bổ sung vào mô hình những phần tử suy diễn được theo quy ước, để người thiết kế chỉ phải khai báo phần mang tính đặc thù. Có bốn phép biến đổi, trong đó phép thứ tư là trọng tâm của luận văn.

3.5.1 Ba phép biến đổi trên khía cạnh cấu trúc

(a) Chuẩn hóa lớp miền. Mỗi lớp thực thể chưa có thuộc tính định danh được bổ sung một định danh; các thuộc tính vết thời gian (thời điểm tạo, thời điểm cập nhật) được thêm theo quy ước. Đây là những thuộc tính mà mọi thực thể đều cần nhưng không mang thông tin thiết kế, nên việc bắt người vẽ khai báo chúng chỉ làm sơ đồ rối thêm.

(b) Sinh thao tác mặc định. Gốc tụ hợp không khai báo phương thức nào được xem là chỉ cần các thao tác cơ bản, và nhận đủ bộ thao tác tạo, đọc theo định danh, đọc danh sách, cập nhật và xóa. Nếu gốc tụ hợp đã khai báo phương thức, phép biến đổi này không can thiệp – nguyên tắc “khai báo tường minh thắng quy ước”.

(c) Sinh kho lưu trữ. Mỗi gốc tụ hợp chưa có kho lưu trữ tương ứng được sinh một giao diện truy xuất, tuân theo khuôn mẫu repository của DDD.

3.5.2 Lan truyền quyền theo phân cấp vai trò

Phép biến đổi thứ tư hiện thực hóa ngữ nghĩa RBAC_1 (Định nghĩa 2.2): tính tập quyền hiệu dụng perm^* cho mọi vai trò. Bài toán không tầm thường vì phân cấp có thể nhiều tầng: quyền của một vai trò ở đáy phải “chảy ngược” lên tất cả các vai trò kế thừa nó, trực tiếp lẫn gián tiếp.

Luận văn sử dụng thuật toán lặp tới điểm bất động, trình bày trong Listing 3.3.

```
Vao : chỉnh sách P = <R, Q, grant, extends, ...>
Ra  : ánh xạ  $\text{perm}^* : R \rightarrow 2^Q$  (tập quyền hiệu dụng)

1  for each r in R do
2     $\text{perm}^*[r] \leftarrow \{ q \mid (r, q) \in \text{grant} \}$       // quyền khai báo trực tiếp
3  end for
4  repeat
5    changed <- false
6    for each (r_con, r_cha) in extends do      // r_con kế thừa r_cha
7      truoc <-  $|\text{perm}^*[r_{\text{con}}]|$ 
8       $\text{perm}^*[r_{\text{con}}] \leftarrow \text{perm}^*[r_{\text{con}}] \cup \text{perm}^*[r_{\text{cha}}]$ 
9      if  $|\text{perm}^*[r_{\text{con}}]| > \text{truoc}$  then changed <- true end if
10   end for
11 until changed = false
12 return  $\text{perm}^*$ 
```

Listing 3.3: Thuật toán lan truyền quyền theo phân cấp vai trò

Tính đúng đắn. Thuật toán tính đúng perm^* theo Định nghĩa 2.2. Thật vậy, gọi $\text{perm}^k[r]$ là giá trị của $\text{perm}^*[r]$ sau vòng lặp thứ k . Bằng quy nạp theo k : sau vòng lặp thứ k , $\text{perm}^k[r]$ chứa mọi quyền của các vai trò cách r không quá k bước trên đồ thị kế thừa. Khi vòng lặp dừng ở bước k^* , không còn quyền mới được thêm, nghĩa là mọi vai trò đạt tới được từ r theo quan hệ *extends* đều đã đóng góp quyền của mình – đúng bằng $\bigcup_{r' \preceq r} \text{perm}^*$ trong định nghĩa.

Tính dừng. Thuật toán luôn kết thúc. Xét đại lượng $\Phi = \sum_{r \in \mathcal{R}} |\text{perm}^*[r]|$. Mỗi vòng lặp hoặc làm Φ tăng thực sự (khi *changed* nhận giá trị đúng), hoặc kết thúc vòng lặp. Vì Φ bị chặn trên bởi $|\mathcal{R}| \times |\mathcal{Q}|$ – một số hữu hạn – nên số vòng lặp không vượt quá $|\mathcal{R}| \times |\mathcal{Q}|$. Đây là lập luận điểm bất động cổ điển trên một dãy hữu hạn với hàm đơn điệu tăng.

Độ phức tạp. Mỗi vòng lặp duyệt qua $|\text{extends}|$ cạnh, mỗi lần hợp tập tốn $O(|\mathcal{Q}|)$. Số vòng lặp bị chặn bởi chiều cao h của đồ thị kế thừa cộng một. Vậy độ phức tạp là $O(h \cdot |\text{extends}| \cdot |\mathcal{Q}|)$. Trong thực tế h rất nhỏ (thường không quá 3–4 tầng), nên chi phí không đáng kể.

Điều kiện tiên quyết. Lập luận về tính đúng đắn giả định đồ thị kế thừa không có chu trình. Nếu có chu trình, thuật toán vẫn dừng (lập luận về Φ không dùng tới giả thiết này) nhưng kết quả mất ý nghĩa: mọi vai trò trong chu trình sẽ có cùng tập quyền, tức

phân cấp trở nên vô nghĩa. Vì vậy bất biến I5 phải được kiểm tra – và trong quy trình, việc này diễn ra ở bước 4, ngay sau bước 3.

Ví dụ 3.1. Xét ba vai trò với quan hệ *Admin* → *Officer* → *Staff* và các quyền khai báo trực tiếp: *Staff* có VIEW_STUDENTS, *Officer* có ENROLL_STUDENTS, *Admin* có MANAGE_STUDENTS. Sau vòng lặp thứ nhất, *Officer* nhận thêm VIEW_STUDENTS và *Admin* nhận thêm ENROLL_STUDENTS. Sau vòng lặp thứ hai, *Admin* nhận tiếp VIEW_STUDENTS (đến từ *Staff* qua *Officer*). Vòng lặp thứ ba không thêm gì và thuật toán dừng. Ví dụ này cho thấy vì sao một lượt duyệt là không đủ: quyền của *Staff* cần hai vòng mới tới được *Admin*.

3.6 Bước 4 - Kiểm chứng ràng buộc

3.6.1 Quy trình đánh giá

Bước 4 đánh giá tập bất biến của Bảng 3.2 trên mô hình đã làm giàu. Với mỗi bất biến thuộc nhóm “thiết kế”, quy trình gồm ba việc: xác định tập đối tượng thuộc ngữ cảnh của bất biến; đánh giá biểu thức trên từng đối tượng; và với mỗi trường hợp sai, tạo một bản ghi chẩn đoán.

Bản ghi chẩn đoán gồm bốn thành phần: mã và tên bất biến bị vi phạm; phần tử mô hình gây vi phạm; vị trí xuất xứ (tập, dòng) lấy từ thông tin do bước 1 mang theo; và mô tả tình huống vi phạm ở dạng người đọc hiểu được. Thành phần thứ ba là điểm khiến chẩn đoán trở nên hữu dụng trong thực tế: người thiết kế được dẫn thẳng tới dòng cần sửa.

3.6.2 Ranh giới giữa hai thời điểm kiểm chứng

Không phải bất biến nào cũng quyết định được ở mức mô hình. Bảng 3.2 phân định như sau:

- **Kiểm ở thời điểm thiết kế** (I1–I5, và phần tĩnh của I6): các bất biến chỉ phụ thuộc vào bản thân đặc tả. Vi phạm ở nhóm này khiến quy trình dừng, không sinh mã. Đây là giá trị cốt lõi mà phương pháp mang lại so với cách cài đặt thủ công: một chính sách mâu thuẫn bị chặn từ khi còn là bản vẽ.
- **Kiểm ở thời điểm vận hành** (I7, I8, và phần động của I6): các bất biến phụ thuộc trạng thái hệ thống lúc chạy – phiên đăng nhập nào đang mở, bao nhiêu người dùng đang giữ vai trò. Với nhóm này, bước 4 không phán quyết mà *ghi nhận yêu cầu kiểm tra*, để bước 5 sinh chúng thành các điểm kiểm soát trong mã.

Nhận xét 3.3. Ranh giới này không phải là hạn chế kỹ thuật mà là hệ quả logic: một mệnh đề nói về trạng thái phiên làm việc đơn giản là không có giá trị chân lý khi hệ thống chưa chạy. Điều phương pháp bảo đảm được là: mọi bất biến *quyết định được* ở mức mô hình đều đã được kiểm; và mọi bất biến *không quyết định được* ở mức mô hình đều được sinh thành mã kiểm tra, chứ không bị bỏ quên.

3.7 Bước 5 - Sinh mã theo mẫu

3.7.1 Ánh xạ khái niệm sang sản phẩm mã

Bước 5 biến mô hình hợp lệ thành mã nguồn. Nguyên tắc là ánh xạ có hệ thống: mỗi khái niệm của mô hình tương ứng một loại sản phẩm mã, theo Bảng 3.3.

Bảng 3.3: Ánh xạ khái niệm mô hình sang sản phẩm mã

Khái niệm mô hình	Sản phẩm mã
Ngữ cảnh giới hạn	Một mô-đun ứng dụng phân tầng riêng
Lớp thực thể, đối tượng giá trị, sự kiện miền	Lớp mã ở tầng miền
Gốc tụ hợp	Lớp mã ở tầng miền, kèm các thao tác đã làm giàu ở bước 3
Kho lưu trữ	Giao diện ở tầng miền và hiện thực ở tầng hạ tầng
Ca sử dụng (kèm chuỗi hành động AGL)	Đơn vị xử lý ở tầng ứng dụng, thân phương thức phác theo chuỗi hành động
Tập vai trò	Kiểu liệt kê vai trò
Tập quyền	Kiểu liệt kê quyền
Quan hệ gán quyền (sau lan truyền)	Bảng gán quyền tra cứu lúc vận hành
Phân cấp vai trò	Bảng phân cấp phục vụ mở rộng quyền lúc vận hành
Bất biến nhóm vận hành (I6-I8)	Các phép kiểm tra bên trong điểm kiểm soát truy cập
Quyền yêu cầu của một thao tác	Siêu dữ liệu gắn trên điểm cuối tương ứng

3.7.2 Ba nguyên tắc của cơ chế sinh

(a) Mẫu sinh không chứa logic. Mọi tính toán – bao đóng quyền, danh sách phụ thuộc, quy tắc đặt tên – được thực hiện trong một bước chuẩn bị dữ liệu, tạo ra một cấu trúc phẳng chỉ gồm chuỗi và danh sách. Mẫu sinh chỉ điền dữ liệu vào khuôn. Lợi ích: logic sinh mã kiểm thử được bằng kiểm thử đơn vị thông thường, thay vì phải so khớp văn bản kết xuất; và mẫu đủ đơn giản để người không viết ra nó vẫn đọc hiểu.

(b) Chú thích truy vết. Mỗi tệp sinh ra mang một khối chú thích ghi rõ nó được sinh từ phần tử mô hình nào và từ dòng nào của tài liệu đặc tả. Đây là chiều còn lại của quan hệ truy vết hai chiều: từ mô hình xuống mã đã có nhờ quy trình sinh, từ mã ngược lên mô hình có nhờ chú thích này.

(c) Sinh lại an toàn. Bước sinh mã có tính idempotent: với cùng một đặc tả đầu vào, kết quả sinh ra là như nhau ở mọi lần chạy. Nhờ đó việc sinh lại sau khi sửa mô hình tạo ra một khác biệt *đúng bằng* phần đã thay đổi, thuận lợi cho việc rà soát. Đối với phần mã do lập trình viên bổ sung, phương pháp dùng cơ chế vùng được bảo vệ: các đoạn nằm giữa cặp dấu hiệu quy ước được trích ra trước khi sinh và chèn lại sau khi sinh.

3.8 Các tính chất của phương pháp

Mục này phân tích bốn tính chất mà phương pháp bảo đảm, tương ứng bốn yêu cầu “sao cho” trong phát biểu bài toán ở mục 1.4.

Tính trực giao. Khía cạnh phân quyền nằm trên một nhánh riêng của mô hình hợp nhất và chỉ liên hệ với khía cạnh cấu trúc qua một loại tham chiếu duy nhất (quyền $\rightarrow$ lớp miễn). Hệ quả: loại bỏ toàn bộ nhánh $\mathcal{P}$ khỏi mô hình vẫn cho một mô hình hợp lệ và sinh được mã – chỉ khác là mã không có lớp phân quyền. Ở chiều ngược lại, thêm hoặc sửa chính sách không làm thay đổi mã sinh ra cho tầng miễn. Tính chất này được kiểm chứng thực nghiệm ở Chương 4.

Tính truy vết hai chiều. Chiều xuôi được bảo đảm bởi bản thân quy trình sinh: mọi phần tử mã đều có nguồn gốc từ một phần tử mô hình. Chiều ngược được bảo đảm bởi chủ thích truy vết, dựa trên thông tin vị trí nguồn mà bước 1 gắn vào và các bước sau bảo toàn.

Tính tự động và lặp lại được. Toàn bộ năm bước chạy trong một lượt, không cần can thiệp thủ công ở giữa. Kết hợp với tính idempotent của bước 5, quy trình có thể chạy lại bất kỳ lúc nào; mô hình luôn là nguồn sự thật duy nhất.

Tính độc lập với công nghệ. Bốn bước đầu chỉ thao tác trên các khái niệm mô hình hóa: lớp, quan hệ, vai trò, quyền, bất biến. Tri thức về nền tảng đích tập trung hoàn toàn trong bộ mẫu của bước 5. Do đó, việc chuyển sang một nền tảng đích khác về nguyên tắc chỉ đòi hỏi viết bộ mẫu mới, giữ nguyên bốn bước đầu và toàn bộ metamodel. Cần lưu ý đây là lập luận dựa trên thiết kế; việc kiểm chứng nó bằng thực nghiệm trên một nền tảng thứ hai thuộc hướng phát triển và được nêu lại trong phần phân tích nguy cơ ảnh hưởng tính hợp lệ ở Chương 4.

3.9 Tổng kết chương

Chương này đã trình bày phương pháp đề xuất theo hai phần. Phần thứ nhất định nghĩa ngôn ngữ đặc tả chính sách RBAC với đầy đủ ba thành phần: cú pháp trừu tượng là metamodel bảy khái niệm (Định nghĩa 3.1); cú pháp cụ thể là profile trên sơ đồ lớp với văn phạm quyền có cấu trúc; ngữ nghĩa tĩnh là tám bất biến phân theo ba cấp RBAC. Phần thứ hai trình bày quy trình năm bước, trong đó đáng chú ý là thuật toán lan truyền quyền với chứng minh tính đúng đắn và tính dừng (mục 3.5), và nguyên tắc phân định hai thời điểm kiểm chứng (mục 3.6). Chương tiếp theo trình bày công cụ hiện thực hóa phương pháp và kết quả đánh giá thực nghiệm.

Tài liệu tham khảo

- (2014). *Object Constraint Language (OCL), Version 2.4*. Object Management Group. Document formal/2014-02-03. <https://www.omg.org/spec/OCL/2.4>.
- (2017). *OMG Unified Modeling Language (OMG UML), Version 2.5.1*. Object Management Group. Document formal/2017-12-05. <https://www.omg.org/spec/UML/2.5.1>.
- (2025). *NestJS Documentation: Guards, Custom Decorators, Authorization, CQRS*. NestJS. <https://docs.nestjs.com>.
- (2025). *PlantUML Language Reference Guide*. PlantUML. <https://plantuml.com/guide>.
- Basin, D., Doser, J., and Lodderstedt, T. (2006). Model driven security: From UML models to access control infrastructures. *ACM Transactions on Software Engineering and Methodology*, 15(1):39–91.
- Dang, D.-H., Le, D. M., and Le, V.-V. (2023). AGL: Incorporating behavioral aspects into domain-driven design. *Information and Software Technology*, 163:107284.
- Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.
- Fowler, M. (2010). *Domain-Specific Languages*. Addison-Wesley.
- Gogolla, M., Büttner, F., and Richters, M. (2007). USE: A UML-based specification environment for validating UML and OCL. *Science of Computer Programming*, 69(1–3):27–34.
- Le, D. M. and Dang, D.-H. (2025). A method for composing concerns into a unified domain model in domain-driven design. In *Proceedings of the International Conference on Knowledge and Systems Engineering (KSE 2025)*. IEEE. Cần bổ sung tên đồng tác giả, số trang và DOI theo bản kỷ yếu chính thức.
- Le, D. M., Dang, D.-H., and Nguyen, V.-H. (2018). On domain driven design using annotation-based domain specific language. *Computer Languages, Systems & Structures*, 54:199–235.
- Le, D. M., Dang, D.-H., and Nguyen, V.-H. (2020). Generative software module development for domain-driven design with annotation-based domain specific language. *Information and Software Technology*, 120:106239.

- Le, D. M., Dang, D.-H., and Vo, H. D. (2025). Layered microservices architecture: A multitree-based domain-driven approach. *Information and Software Technology*, 183:107720.
- Le, V.-V., Dang, D.-H., and Le, D. M. (2023). A pattern-based approach to incorporating behavioral aspects into domain models. In *Proceedings of the 15th International Conference on Knowledge and Systems Engineering (KSE 2023)*. IEEE. Cần bổ sung số trang và DOI theo bản kỷ yếu chính thức.
- Li, Y., Du, Z., Fu, Y., and Liu, L. (2022). Role-based access control model for inter-system cross-domain in multi-domain environment. *Applied Sciences*, 12(24):13036.
- Lodderstedt, T., Basin, D., and Doser, J. (2002). SecureUML: A UML-based modeling language for model-driven security. In *UML 2002 — The Unified Modeling Language*, volume 2460 of *Lecture Notes in Computer Science*, pages 426–441. Springer.
- Newman, S. (2021). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media, 2 edition.
- Ray, I., Li, N., France, R., and Kim, D.-K. (2004). Using UML to visualize role-based access control constraints. In *Proceedings of the Ninth ACM Symposium on Access Control Models and Technologies (SACMAT '04)*, pages 115–124. ACM.
- Richardson, C. (2018). *Microservices Patterns: With Examples in Java*. Manning Publications.
- Sandhu, R. S., Coyne, E. J., Feinstein, H. L., and Youman, C. E. (1996). Role-based access control models. *Computer*, 29(2):38–47.
- Vernon, V. (2013). *Implementing Domain-Driven Design*. Addison-Wesley.
- Voelter, M., Benz, S., Dietrich, C., Engelmann, B., Helander, M., Kats, L. C. L., Visser, E., and Wachsmuth, G. (2013). *DSL Engineering: Designing, Implementing and Using Domain-Specific Languages*. CreateSpace Independent Publishing Platform. <http://dslbook.org>.